Prague University of Economics and Business
Faculty of Informatics and Statistics

Applying Semantic Search and Large Language Models to Solve Assortment Gaps in E-Commerce

BACHELOR THESIS

Study program: Data Analytics

Author: Ngoc Khiem Tran
Supervisor: doc. Ing. Ondřej Zamazal, Ph.D.

Prague, 12.5.2025

Acknowledgements

Thanks.

Abstract

This thesis addresses the problem of assortment gaps in e-commerce, with a particular focus on the grocery retail sector. The main objective is to develop a robust and scalable solution for identifying missing or substitute products by leveraging semantic search techniques and large language models (LLMs). To this end, a two-stage system is proposed that combines vector-based information retrieval with LLM-driven classification. The first stage employs dense vector embeddings and cosine similarity to retrieve the top candidate products from the internal catalog based on competitor product descriptions. This semantic search system achieved a high recall of 93.24%, with the correct match appearing in the top five results in over 90% of cases and as the top-ranked result in two-thirds of instances. The second stage introduces a Classification Bot, powered by advanced LLMs such as GPT-4o, Gemini 2.0 Flash, and DeepSeek V3, to assess whether a retrieved candidate is indeed identical. Evaluation of this component reveals performance trade-offs: GPT-4o attained the highest recall (74%) and precision (90%), Gemini 2.0 Flash offered minimal latency and cost, and DeepSeek V3 provided a balanced solution. The system's modular architecture enables customisation based on organisational priorities such as accuracy, cost, or runtime. Overall, this two-step framework effectively bridges the semantic gap in product matching, offering a meaningful contribution to automated catalog integration, data quality improvement, and competitive product analysis in modern e-commerce environments.

Keywords

Large Language Models, Natural Language Processing, Machine Learning, Recurrent Neural Networks, Convolutional Neural Networks, Long Short-Term Memory, European Article Number, Identifier, Information Retrieval, Assortment Gap, Semantic Search, Embedding Models.

Contents

List of Abbreviations

LLMs	Large Language Models
NLP	Natural Language Processing
ML	Machine Learning
RNNs	Recurrent Neural Networks
CNNs	Convolutional Neural Networks
LSTM	Long Short-Term Memory
EAN	European Article Number
ID	Identifier
IR	Information Retrieval

Introduction

The rapid growth of e-commerce has created significant challenges and opportunities for businesses, particularly in the area of product assortment management. One major challenge is identifying and addressing assortment gaps—situations where a business lacks specific products or substitutes that competitors offer. Bridging these gaps is essential for enhancing competitiveness, improving customer satisfaction, and increasing revenue.

In the context of grocery e-commerce, this challenge is even more pronounced due to the vast number of products, their variations, and diverse consumer preferences. Businesses must determine not only whether they offer the same products as their competitors but also whether they have alternatives with similar functionalities or substitute potential. Traditional keyword-based search methods often fall short of capturing the nuanced relationships between products, highlighting the need for more advanced approaches.

This bachelor thesis investigates the use of semantic search and large language models (LLMs) to tackle the problem of identifying assortment gaps in e-commerce. Semantic search utilizes the capabilities of deep learning and natural language processing to comprehend the meaning and context of product descriptions, thereby enabling the discovery of similar or substitute products. By applying embeddings derived from product characteristics, businesses can more effectively compare their assortments with those of competitors.

The main objective of this thesis is to address the challenge of assortment gaps in e-commerce by leveraging the power of semantic search and large language models. To achieve this, the thesis begins with a comprehensive review of existing approaches that utilize deep learning techniques for conducting semantic product search. It then analyzes available product data in the grocery e-commerce domain to identify key characteristics essential for recognising substitutes. Based on this foundation, a semantic search method using vector embeddings is designed and implemented to retrieve the most relevant candidates from the internal catalog. Additionally, the thesis introduces a product classification module using LLMs to determine whether the retrieved items are identical, similar, or different.

The evaluation of the developed system demonstrated strong performance: the semantic search component achieved a recall of over 93%, and the LLM-based classifier, when using GPT-4o, reached a precision of 90%. These results confirm the effectiveness of the proposed two-stage approach in addressing the assortment gap problem in real-world e-commerce scenarios.

1. E-Commerce and Semantic Search

1.1 E-Commerce and Product Assortment Challenges

E-commerce, short for electronic commerce, refers to the buying and selling of goods or services using the internet, as well as the transfer of money and data to execute these transactions. It encompasses a range of business models including Business-to-Consumer (B2C), Business-to-Business (B2B), Consumer-to-Consumer (C2C), and more recently emerging formats such as social commerce and live e-commerce (McKinsey & Company, 2023). With the continuous digitisation of global markets, e-commerce has grown rapidly, transforming how businesses operate and how consumers make purchasing decisions. According to Deng (Deng, 2022), the expansion of e-commerce has not only diversified customer preferences but also redefined the competitive landscape across various industries.

The rapid evolution of e-commerce has fundamentally reshaped both consumer purchasing behaviour and business operations worldwide. Online platforms provide customers with unprecedented access to a wide assortment of products, facilitating more personalised and dynamic shopping experiences. However, this accessibility introduces increasing pressure on businesses to maintain product assortments that are both competitive and aligned with evolving consumer expectations.

In highly saturated digital markets, retailers must frequently adapt their catalogues to address shifting customer demands while also differentiating themselves from competitors. This challenge is compounded by factors such as warehouse limitations, supply chain constraints, high inventory management costs, and difficulties in demand forecasting. In traditional brick-and-mortar retail, physical shelf and floor space often restrict the total number of products that can be displayed. Although e-commerce platforms are not bound by these spatial constraints, they face their own operational limitations, including the costs associated with storing and managing an extensive product assortment (Mantrala et al., 2009).

A key concern in digital assortment planning is striking the right balance between variety and efficiency. While a broader product range can increase market coverage, it may also lead to *SKU proliferation*—the uncontrolled increase in stock-keeping units. This can result in increased complexity, poor inventory visibility, deadstock accumulation, and higher carrying costs (ShipBob, 2023). Retailers must navigate these challenges carefully to ensure that the diversity of their offerings does not compromise overall operational performance.

Furthermore, excessive choice can negatively affect customer experience. Although variety is generally desirable, an overly large and uncurated selection may lead to what is known as the “agony of choice”, where consumers experience confusion and indecision due to decision fatigue. Research in behavioural economics has shown that an overwhelming number of options can reduce customer satisfaction and purchasing likelihood (**iyengar2000**). This

phenomenon underscores the need for e-commerce platforms to optimise not just the quantity, but also the relevance and clarity of their product assortments.

In summary, assortment planning in the digital commerce environment involves navigating a multifaceted set of challenges, including customer expectations, operational scalability, and competitive dynamics. As the complexity of online product offerings increases, businesses must turn to data-driven strategies and intelligent systems—such as semantic search and machine learning-based recommendation engines—to design assortments that are both manageable and profitable.

1.2 Assortment Gap in E-Commerce

The concept of an *assortment gap* in e-commerce refers to the absence of certain products or suitable substitutes within a retailer's product catalogue compared to the offerings of its competitors. This gap can lead to decreased customer satisfaction and missed revenue opportunities, as consumers may turn to rival platforms when their preferred items are unavailable. A lack of product coverage not only weakens a retailer's market position but also prevents it from capitalizing on emerging demand trends and consumer preferences.

Identifying assortment gaps plays a vital role in strategic assortment planning and competitive benchmarking. By examining competitors' product ranges, retailers can determine which products are missing from their own inventory. This process can highlight trending or high-demand items that rivals stock but are absent from the business's own offerings, thereby revealing untapped sales potential (Price2Spy, 2023a).

Furthermore, assortment benchmarking offers insight into items that a retailer carries exclusively. These unique products can become a competitive advantage, allowing the retailer to adopt differentiated pricing strategies and capture higher profit margins. In this context, assortment analysis also supports pricing optimisation. If competitors offer comparable products at lower prices or bundle them differently, retailers risk losing market share unless they adapt their pricing structures accordingly.

Despite its strategic importance, detecting assortment gaps is a challenging task due to inconsistent product descriptions, varying naming conventions, and differences in brand representation. Two products with identical functionality may appear different in textual descriptions or metadata, complicating automated comparison. Consequently, advanced tools such as semantic search and product matching systems have become increasingly valuable in bridging these gaps and enabling more accurate inventory assessments.

1.3 Advances in Semantic Search and NLP

Semantic search refers to the process of retrieving information based on the meaning and context of a query rather than relying on exact keyword matches. Unlike traditional search engines, which often return results based on literal string overlap, semantic search systems attempt to understand the intent behind a query and match it with conceptually similar content. This is particularly important in e-commerce, where product names and descriptions can vary across vendors despite representing identical or similar items (Tóth et al., 2022).

Modern semantic search systems leverage advances in Natural Language Processing (NLP) and Large Language Models (LLMs) to perform this type of intelligent matching. These technologies enable machines to interpret and compare unstructured textual data with high accuracy, thus playing a key role in resolving assortment gaps without relying solely on structured identifiers like EAN codes.

1.3.1 Natural Language Processing

Natural Language Processing is a subfield of artificial intelligence that enables computers to understand, interpret, and generate human language. According to Google Cloud, "As a branch of artificial intelligence, NLP uses machine learning to process and interpret text and data. Natural language recognition and natural language generation are types of NLP" (Google Cloud, 2024). NLP plays a foundational role in many modern applications, including voice assistants, machine translation, and semantic search engines.

A typical NLP pipeline involves multiple stages: tokenization, part-of-speech tagging, syntactic parsing, named entity recognition, and semantic analysis (Jurafsky & Martin, 2023). These processes collectively allow systems to derive structured meaning from unstructured text.

In the context of e-commerce, textual product metadata such as titles, descriptions, and brand names are key features for understanding item semantics. As demonstrated in a study on recommendation systems by Wang et al. (Wang et al., 2018), the combination of these attributes can be used to compute semantic similarities between items and enhance the quality of retrieval and recommendation tasks. Specifically, the authors proposed extracting semantic knowledge from item metadata (title, description, and brand) and combining it with customer behavior patterns to improve product discovery and similarity estimation.

Word Embeddings and Semantic Representation To numerically represent text data, NLP systems rely on **word embeddings**. Word embeddings are dense vector representations of words that capture their meanings, contexts, and relationships based on usage in large text corpora. As explained by Turing.com, "Word embeddings convert words into fixed-length, continuous vector representations based on their semantic context, allowing machines to

detect similarities and relationships between words" (Turing.com, 2023). These embeddings map words with similar meanings to nearby points in a high-dimensional space, thus enabling semantic search and comparison.

Example: In an embedding space, the vectors for the words *"king"* and *"queen"* will be close together, just as *"cat"* and *"kitten"* may form a similar cluster.

Several techniques have been developed to create word embeddings:

- **Word2Vec:** A neural network-based method introduced by Mikolov et al. (Goldberg & Levy, 2014), which learns embeddings by predicting words from their surrounding context (CBOW) or vice versa (Skip-gram). It captures deep semantic information by learning word relationships based on their co-occurrence.
- **Transformer-based Embeddings:** More advanced approaches such as BERT (Koroteev, 2021) and multilingual-e5 encode entire sentences or documents instead of individual words. These models are especially useful for tasks involving sentence-level understanding, such as semantic search and classification.

Semantic Search and Similarity Computation

The semantic search component is responsible for identifying products in the internal catalogue that are most semantically similar to a given competitor product query. This process relies on comparing high-dimensional vector representations (embeddings) of product information generated by transformer-based models. The comparison is performed using **cosine similarity**, a widely used metric in natural language processing and information retrieval tasks.

Cosine Similarity Cosine similarity (DataStax, 2023) measures the cosine of the angle between two vectors in an embedding space. It captures how similar two vectors are in terms of direction, regardless of their magnitude. This makes it particularly suitable for semantic comparison, where absolute values of embeddings may vary, but relative orientation holds semantic meaning.

The formula for cosine similarity between two vectors $\mathbf{A}$ and $\mathbf{B}$ is:

$$\text{cosine\_similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

Where:

- $\mathbf{A}$ and $\mathbf{B}$ are the embedding vectors of the query and candidate product.
- $\mathbf{A} \cdot \mathbf{B}$ is the dot product of the vectors.
- $\|\mathbf{A}\|$ and $\|\mathbf{B}\|$ represent the Euclidean norms (magnitudes).

Cosine similarity returns values between -1 and 1, where:

- 1 indicates that the vectors are identical in direction (perfectly similar),
- 0 indicates orthogonality (no similarity),
- -1 indicates opposite direction (completely dissimilar).

In the context of this thesis, cosine similarity allows for the effective retrieval of products that are semantically similar in meaning but may differ in wording or formatting. For instance, the product titles *"Red Bull Energy Drink 0.25L"* and *"Red Bull 250ml Can"* will yield high similarity scores due to their semantic closeness, even though the textual tokens differ.

1.3.2 Large Language Models

According to Raiaan et al., "Large Language Models are a category of language models that utilise neural networks comprising billions of parameters. These models are trained on massive volumes of unlabelled text data through self-supervised learning, which enables them to learn complex linguistic patterns, subtle semantics, and contextual relationships without the need for manually annotated corpora" (Raiaan et al., 2024).

In contrast to earlier sequential models such as recurrent neural networks (RNNs), which process tokens one at a time, transformer-based LLMs operate on entire sequences in parallel. This architectural shift, introduced by the Transformer model (Vaswani et al., 2017), enables significant improvements in both training efficiency and model scalability. As a result, modern LLMs are capable of being scaled up to hundreds of billions of parameters and trained on data sources such as Wikipedia, Common Crawl, and other publicly available web corpora.

A key advantage of transformer-based LLMs is their ability to perform self-supervised learning. This allows them to infer syntactic structures, semantic nuances, and even world knowledge directly from data, enhancing their generalisability and transferability across diverse NLP tasks. In the context of semantic product search, which is the focus of this thesis, LLMs play a crucial role in generating contextual embeddings and understanding user queries at a deeper semantic level. Their capability to encode rich representations of both product descriptions and user intent forms the backbone of modern semantic retrieval systems.

Why Are Large Language Models Important? Large Language Models have become foundational in modern NLP due to their ability to model complex linguistic patterns, capture contextual dependencies, and generate semantically coherent text. These models enable significant advances in a range of downstream applications, including semantic search, machine translation, question answering, summarisation, and automated content generation. Their scalability, multilingual capabilities, and adaptability to diverse domains make them particularly well-suited for large-scale, real-world deployments. As Raiaan et al. note, "LLMs have become the cornerstone of many state-of-the-art NLP systems, demonstrating superior performance across various benchmarks and real-world use cases" (Raiaan et al., 2024). Their

emergence has shifted the paradigm from task-specific models to general-purpose language models that can be fine-tuned for specific applications with relatively little supervision.

Use of LLMs in This Thesis In this thesis, LLMs play a central role in enhancing product matching and classification:

- They are used to generate high-quality product embeddings, which encode semantic information about product names, descriptions, and brand attributes.
- An LLM-powered **Classification Bot** is developed to analyse the results returned by the semantic search system and categorise products into **Identical**, **Similar**, or **Different** (see Section 3.6).
- This approach reduces the need for manual product matching, which is costly and time-consuming, and addresses cases where exact identifiers such as EAN codes are missing.

By leveraging LLMs such as GPT-4o (OpenAI, 2024), DeepSeek V3 (DeepSeek AI, 2024), and Gemini 2.0 Flash (Google DeepMind, 2024), the system demonstrates high adaptability, precision, and efficiency in solving the assortment gap problem in e-commerce.

2. Semantic Search of Product and Deep Learning

2.1 Introduction to Semantic Product Search

Online retail platforms have become ubiquitous, offering consumers access to vast catalogs containing millions, and sometimes billions, of products. Within this digital landscape, the search function serves as a primary gateway for users to discover items that meet their needs (Li et al., 2020). Traditionally, product search engines have relied heavily on lexical matching techniques, primarily comparing keywords in a user's query against terms present in product titles, descriptions, and metadata. This approach, rooted in classic Information Retrieval (IR) models such as TF-IDF and BM25, works reasonably well for specific, unambiguous queries where users know the exact terminology (Manning et al., 2008).

However, keyword-based search suffers from fundamental limitations, often referred to as the *vocabulary mismatch problem* or the broader *semantic gap* (Kuzi et al., 2017). Users frequently employ different terms than those used in product listings (e.g., "running shoes" versus "trainers", or "cell phone" versus "smartphone"). Additionally, user queries may be ambiguous, incomplete, or express complex needs beyond simple keyword presence (e.g., "warm jacket for skiing" or "adapter for MacBook Pro to HDMI"). Lexical matching systems struggle to grasp the underlying intent of the query and the semantic meaning of product attributes, leading to issues such as missed relevant items (low recall) or the retrieval of irrelevant items (low precision).

Semantic product search aims to overcome these limitations by moving beyond surface-level text matching to understanding the deeper meaning and intent encoded in both user queries and product information (Zhang et al., 2020). The core objective is to retrieve products that are conceptually relevant to the user's need, even if the query does not exactly match the product text. This requires representing queries and products in ways that capture their semantic relationships, often leveraging techniques from Natural Language Processing (NLP) and Machine Learning (ML).

Successful semantic search can significantly enhance retrieval accuracy, improve product discoverability, and ultimately contribute to a more satisfying user experience and higher conversion rates for e-commerce platforms (Price2Spy, 2023b). Semantic understanding is particularly vital in modern online shopping environments, where customer expectations for personalised and accurate recommendations are higher than ever.

Nevertheless, addressing the challenge of semantic product search is non-trivial. It involves managing the enormous scale and dynamic nature of product catalogs, dealing with linguistic ambiguity, understanding implicit and latent product attributes, and potentially integrating

multimodal information such as product images. This chapter provides a review of the computational approaches developed to address these challenges. We will first briefly examine traditional machine learning techniques that laid the early groundwork, followed by an exploration of modern deep learning methods, which have demonstrated significant promise in bridging the semantic gap. Furthermore, we will summarise the key characteristics and developments of these approaches in the context of semantic product search, highlighting their potential and commonly reported strengths based on insights from the relevant literature.

2.2 Traditional Machine Learning Approaches

To address the limitations of keyword-based search mentioned previously, traditional Machine Learning (ML) techniques were employed to incorporate basic statistical and semantic patterns.

Early methods such as **Vector Space Models** (VSMs), including **TF-IDF**, primarily captured term importance but struggled with challenges such as synonymy and polysemy (Manning et al., 2008). **Latent Semantic Analysis** (LSA) was introduced to uncover latent semantic structures within text by applying singular value decomposition (SVD) on term-product matrices, allowing for query-product comparison in a reduced vector space (Wiemer-Hastings, 2004). Although LSA provided an improvement over purely lexical methods, its reliance on linear assumptions and its computational intensity limited its effectiveness, particularly in large-scale product datasets.

Probabilistic models such as **Okapi BM25** provided more robust term weighting compared to basic TF-IDF, better accounting for term frequency saturation and document length normalization (Robertson & Zaragoza, 2009). In parallel, topic modeling approaches like **Latent Dirichlet Allocation** (LDA) were applied to uncover underlying themes or topics across product corpora (Blei et al., 2003). However, LDA was more suited for general topic discovery rather than fine-grained semantic matching at the individual product level.

A significant advance came with the introduction of **Learning to Rank** (LTR) frameworks, which sought to directly optimize ranking metrics such as Normalized Discounted Cumulative Gain (NDCG) or Mean Average Precision (MAP) (Liu, 2009). Early LTR models, such as `SVM-Rank` and `LambdaMART`, effectively combined features from lexical models (e.g., BM25), product metadata, and user interaction signals to learn ranking functions (Chapelle & Chang, 2011). Nevertheless, these approaches heavily depended on manual feature engineering and failed to automatically capture deep semantic relations between products and queries.

While traditional methods demonstrated meaningful improvements over basic keyword-based search, their performance was typically assessed using rank-aware Information Retrieval (IR) metrics such as Precision@k, Recall@k, and Normalized Discounted Cumulative Gain (NDCG@k). These metrics focus on evaluating the relevance of the top-ranked results returned by a system, aligning more closely with user satisfaction in practical search applica-

tions (Wiemer-Hastings, 2004). Nevertheless, the shallow textual representations and reliance on handcrafted features in these methods proved inadequate to fully bridge the semantic gap between user intent and product descriptions. This shortfall led to the adoption of deep learning approaches, which are discussed in the following section.

2.3 Deep Learning Methods for Semantic Product Search

The limitations of traditional ML approaches, particularly their inability to automatically learn complex, hierarchical representations of meaning from raw data, motivated the adoption of Deep Learning (DL) techniques for semantic product search. Deep Learning models, characterized by multiple layers of non-linear transformations, excel at capturing intricate patterns and semantic nuances in text, images, and user behavior data (Henaff et al., 2015). This section reviews key DL architectures applied to product search, emphasizing their mechanisms and reported performance.

Embedding-Based Methods: Learning Semantic Representations

A cornerstone of DL for semantic search is the concept of **embeddings**: dense, low-dimensional vector representations of entities such as words, sentences, queries, or products. Unlike sparse representations (e.g., one-hot encoding, TF-IDF), embeddings position semantically similar entities closer together in vector space (Mikolov et al., 2013).

Early methods adapted general-purpose word embeddings like Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) for product search, often by averaging word vectors in product descriptions or queries. More effective approaches directly learn embeddings tailored for query-product matching using user interaction data (e.g., clicks, purchases) (Zhang et al., 2020).

Siamese Networks: These architectures process the query and candidate products independently through identical subnetworks with shared weights (Bromley et al., 1994). Relevance is computed via a similarity function (e.g., cosine similarity) between the resulting embeddings. Training typically employs contrastive loss or triplet loss to bring matching pairs closer while pushing apart mismatches (Schroff et al., 2015). Empirical results demonstrate significant improvements in Recall@k and NDCG@k in real-world deployments by companies such as Amazon, Alibaba, and Walmart.

Two-Tower Models: A common industry implementation of Siamese networks involves separate "towers" for query and product encoding (Covington et al., 2016). Pre-computing product embeddings enables fast nearest neighbor search at inference. This architecture has proven highly scalable for large catalogs.

Sequence Models (RNNs, LSTMs) and Convolutional Models (CNNs)

Prior to the Transformer revolution, sequential models like Recurrent Neural Networks - (RNNs) and their variants, `Long Short-Term Memory` (LSTM) (Hochreiter & Schmidhuber, 1997) and `Gated Recurrent Units` (GRU) (Cho et al., 2014), were widely used to encode text inputs in product search systems. These models captured word order and long-range dependencies.

Convolutional Neural Networks (CNNs), originally developed for image processing, were adapted for text by applying filters across sequences of word embeddings (Kim, 2014). Text-CNNs effectively detected key phrases indicative of relevance. Both LSTMs and CNNs were commonly employed within Siamese or Two-Tower architectures for product search tasks.

Transformer Models: The Rise of Attention

The introduction of the **Transformer** architecture (Vaswani et al., 2017) revolutionized NLP and semantic search. Transformers capture long-range dependencies and contextual nuances with self-attention mechanisms. Pre-trained models like `BERT` (**devlin2019bert**) and its variants (e.g., RoBERTa, ALBERT) offer powerful contextualized embeddings that can be fine-tuned for downstream tasks.

In the bi-encoder architecture, the term "bi" refers to the use of two independent encoders—typically based on Transformer models—to process the query and product texts separately. Each encoder maps its respective input into a dense semantic vector space. One popular implementation of this approach is Sentence-BERT, which adapts BERT for efficient sentence-level embeddings (Reimers and Gurevych, 2019). Because product embeddings can be pre-computed offline, this architecture supports scalable retrieval through approximate nearest neighbor search, making it well-suited for large e-commerce catalogs. Although bi-encoders do not model fine-grained interactions between queries and documents, they have been shown to achieve competitive retrieval performance on a variety of benchmarks.

In contrast, the cross-encoder architecture processes the query and product text jointly by concatenating them and passing the combined sequence through a single Transformer model. This setup allows the model to perform deep token-level attention across both inputs, capturing intricate semantic relationships (Nogueira and Cho, 2019). As a result, cross-encoders typically yield more accurate relevance scores compared to bi-encoders. However, their computational cost is significantly higher since encoding must be performed for each query-document pair at inference time. Consequently, cross-encoders are often used in a second-stage re-ranking step, where they refine a smaller set of candidate results retrieved by a faster model such as a bi-encoder.

In summary, Deep Learning methods, particularly those leveraging embeddings and Transformer architectures, have significantly advanced semantic product search. They enable a

deeper understanding of language, user intent, and product attributes, surpassing traditional ML models in both precision and recall. Subsequent sections of this thesis build upon these techniques, implementing and evaluating semantic search systems grounded in these concepts.

3. Solving Assortment Gaps using Semantic Search and LLMs

3.1 Problem Definition

In the context of e-commerce, the **assortment gap problem** (as introduced in Section 1.2) presents a significant challenge for businesses seeking to maintain a competitive and comprehensive product offering. Retailers must continuously ensure they are not missing key products that competitors offer—particularly those that are best-sellers or in high demand. The inability to stock such products can lead to lost revenue opportunities and diminished customer satisfaction.

One approach to mitigating this issue is through **web scraping**, which allows for the automated extraction of product data from competitors' websites. Among the most critical identifiers retrieved through scraping is the **EAN code** (European Article Number), which uniquely identifies products by encoding the manufacturer, packaging configuration, and product type (Prisync, 2023). When available, the EAN code enables direct and precise matching of identical products across internal and competitor catalogs.

However, a growing number of retailers have started removing or obscuring EAN codes from public listings. This shift, often driven by competitive strategy, severely limits the feasibility of automated matching and forces businesses to rely on costly manual review or increasingly sophisticated alternatives. Manual product pairing is not only labour-intensive and error-prone but also unscalable in high-volume e-commerce environments.

To address this limitation, this thesis proposes the development of a **semantic search system**. By employing modern natural language processing and machine learning methods, the system can identify identical or similar products based on textual and contextual cues—such as product name, description, and brand—even in the absence of standard identifiers like EAN. This approach facilitates scalable, efficient, and intelligent assortment analysis, enabling businesses to close gaps in their offerings and respond proactively to market demands.

3.2 Methodology

This section presents the complete workflow used to address the assortment gap problem, covering all stages from data collection to evaluation. The methodology is divided into several key steps, each aligned with the respective implementation details described in later chapters:

1. **Data Collection, Preprocessing, and Analysis**

Product data are collected from both internal systems and competitor websites. The collected attributes include product names, brand names, descriptions, units, and, when available, EAN codes. These data are cleaned to remove duplicates, standardised to ensure consistency, and analysed to identify patterns and key features essential for product comparison (see Section 3.3). The preprocessing stage also includes generating missing product descriptions using a custom LLM bot and normalising product names.

2. **Embedding Generation and Vector Storage**
   To enable semantic comparison across product catalogs, each product is first transformed into a structured textual format by concatenating selected attributes such as brand name and product description. This processed text is then encoded into a high-dimensional vector using the `intfloat/multilingual-e5-large` transformer model, accessible via the Hugging Face library[1]. This multilingual model captures nuanced semantic relationships, making it well-suited for cross-lingual product matching tasks (see Section 3.4).
   The resulting embedding vectors are subsequently stored in a PostgreSQL database extended with the `pgvector` module.
3. **Semantic Search System**
   A semantic search engine is implemented to retrieve the top 10 most semantically similar internal products for each competitor product. The query is embedded using the same model, and cosine similarity is used to rank results (see Section 3.5). This process enables approximate matching even when no shared identifiers such as EAN codes are available.
4. **LLM-based Product Classification**
   To further improve product identification accuracy, a Classification Bot based on large language models (LLMs) is introduced (see Section 3.6). Using a structured prompt and comparison logic, the model evaluates retrieved product candidates and classifies them as **Identical**, **Similar**, or **Different**. This approach compensates for limitations in purely embedding-based similarity.
5. **Evaluation of the Semantic Search System**
   The performance of the semantic search system is evaluated using ground-truth data with known EAN codes (see Section 4.1). The evaluation checks whether the exact internal product appears in the top 10 retrieved items. The results are summarised using accuracy metrics, specifically focusing on whether the matching product is ranked in the top 10, top 5, or top 1 positions.
6. **Evaluation of the Classification Bot**
   The Classification Bot is evaluated using the same merged dataset. Its predictions are compared against ground-truth matches derived from EAN code alignment. Two metrics are computed: **accuracy**, which reflects how often the bot correctly identifies the identical product, and **precision**, which measures how often products labelled as `identical` are indeed correct (see Section 4.1).

This methodology provides a modular and scalable approach to identifying assortment gaps,

[1] https://huggingface.co/intfloat/multilingual-e5-large

even when explicit identifiers such as EAN codes are unavailable. By combining semantic search with LLM-based classification, the system delivers both precision and flexibility for real-world e-commerce product matching tasks.

3.3 Analysis of available data

3.3.1 Data Sources

The data collection process involved two primary sources: (1) *Competition data* and (2) *Internal Product Database*.

Note: The full raw datasets, including additional columns not shown here, are provided as electronic attachments. See Appendix A.

Competition Data The first source was an Excel sheet (Table 3.1) provided monthly by a third-party partner. This dataset contained information on the top best-selling products for a given month, including attributes such as:

- Competitor name
- EAN (European Article Number) code
- Product name
- Product category and subcategory
- Brand and manufacturer
- Monthly sales and unit sales

Products were analyzed to determine whether identical or similar items existed in the internal inventory that could be considered substitutes. An excerpt of the competitor data is provided in Table 3.1.

Brand	Product Name
BERLINER KINDL	BERLINER KINDL JUBILAEUMSPILS PILS O ANGABE ALKOHOLHALTIG 0.5 L RW NORMAL 20
RED BULL	RED BULL NORMAL M CO2 NORMAL 1 0.25 L DS EW
BERLINER PILSNER	BERLINER PILSNER PILS O ANGABE ALKOHOLHALTIG 0.5 L RW NORMAL 20
DALLMAYR	DALLMAYR PRODOMO COFF GEM 500 G PACK
VOLVIC	VOLVIC NATURELLE NORMAL O CO2 NORMAL COFFEINFREI 6 1.5 L EW PETFL

Table 3.1: Excerpt from the competitor data table. *Source: Author.*
(For access to the complete dataset (`Competition_data.xlsx`), refer to Appendix A.)

Internal Product Database The second source was the internal product database, which contained a comprehensive list of products maintained by the company. This structured product table (Table 3.2) included the following attributes:

- Unique Product ID
- Product name and description
- Product category and subcategory
- Brand
- Unit information

This internal data served as the reference for matching and analyzing competitor products. An excerpt from the internal product table is shown in Table 3.2.

Unique ID	**Name**
4100080100938	Berliner Kindl Jubilaums Pilsener Kiste 20x0,5 l (MEHRWEG) (10 l)
90162565	Red Bull Energy Drink (EINWEG) (0.25 l)
4004160105335	Berliner Pilsner Kiste 20x0,5 l (MEHRWEG) (10 l)
4008167103714	Dallmayr prodomo gemahlen (0.5 kg)
3057640186233	Volvic Naturelle 6x1,5 l PET (EINWEG) (9 l)

Table 3.2: Excerpt from the internal product table. *Source: Author.*
(The complete internal product dataset (`Internal_products.csv`), can be found in the Appendix A)

3.3.2 Cleaning and Transformation

The raw data from both sources presented two key challenges that required preprocessing to ensure suitability for analysis: (1) *Cryptic Product Names* and (2) *Lack of Product Descriptions.*

1. Cryptic Product Names: Product names in the competitor dataset were often cryptic, making direct comparisons difficult. For example:

- *"BERLINER KINDL JUBILAEUMSPILS PILS O ANGABE ALKOHOLHALTIG 0.5 L RW NORMAL 20"* was cryptic and verbose.
- *"RED BULL NORMAL M CO2 NORMAL 1 0.25 L DS EW"* lacked clarity for product identification.

These examples highlight the need for standardization before further analysis.

2. Lack of Product Descriptions: The competitor dataset did not include product descriptions, which are essential for embedding and vectorization processes. Descriptions provide semantic richness by combining information from product names and other attributes, such as brand and category.

To address these challenges, two custom bots (Name Standardization Bot and Description Generation Bot) were developed using the GPT-4-o model (OpenAI, 2024), as described below:

Name Standardization Bot: The Name Standardization Bot was developed to transform cryptic and verbose competitor product names into clear, concise, and standardized formats that align with the company's naming conventions. The bot was built using a few-shot learning approach, leveraging examples provided in an Excel file with two columns: *Competitor Product Name* and *Our Product Name*. These examples served as a guide for the bot to learn the naming conventions and perform accurate standardization.

For the complete source code (`NameStandardizationBot.py`) and the prompt template, see Appendix A

Prompt Design: The bot was guided by the following prompt:

The full prompt template is available here Prompt template for the Name standardization Bot

Few shot Example: The few shot examples provided to the bot were stored in an Excel file containing two columns: *Competitor Product Name* and *Our Product Name*. An excerpt of these examples is shown in Table 3.3, which demonstrates how competitor product names were mapped to their standardized equivalents:

Competitor Product Name	**Our Product Name**
BERLINER KINDL JUBILAEUMSPILS PILS O ANGABE ALKOHOLHALTIG 0.5 L RW NORMAL 20	Berliner Kindl Jubiläumspils Pilsener Kiste 20x0,5 l (MEHRWEG) (10 l)
COCACOLA ZERO SUGAR LEMON 1,5L	Coca-Cola Zero Sugar Lemon Flasche 1,5 l
NIVEA CREME BLUE CLASSIC 400ML	Nivea Creme Classic Dose 400 ml

Table 3.3: Excerpt of examples provided to the bot for few-shot learning. *Source: Author.* (For the complete Few shot data (`FewShotExamples.xlsx`) see Appendix A)

Using these examples, the bot was tasked to standardize competitor product names, ensuring they matched the company's naming conventions.

Functionality: The bot was designed to:

- **Standardize naming style:** Align product names with the internal database by adhering to predefined capitalization, word order, and terminology conventions.
- **Preserve essential details:** Include critical attributes such as product type, size, and packaging format, ensuring comprehensive product identification.
- **Eliminate unnecessary information:** Remove redundant or unclear terms like *NORMAL* that do not contribute to product differentiation.
- **Harmonize terminology:** Replace inconsistent phrasing for uniformity across product categories.
- **Enhance clarity:** Ensure the standardized name is unambiguous and easily understood by both internal stakeholders and customers.

Example Transformations: Below are additional sample transformations performed by

the bot:

- **Input:** BERLINER KINDL JUBILAEUMSPILS PILS O ANGABE -ALKOHOLHALTIG 0.5 L RW NORMAL 20
- **Output:** Berliner Kindl Jubiläumspils Pilsener Kiste 20x0,5 l (MEHRWEG) (10 l)
- **Input:** RED BULL NORMAL M CO2 NORMAL 1 0.25 L DS EW
- **Output:** Red Bull Energy Drink (EINWEG) (0.25 l)
- **Input:** GORBATSCHOW WODKA 37.5 PRZ 0.7 L
- **Output:** Gorbatschow Wodka (0.7 l)

This few-shot learning approach ensured that the bot systematically aligned competitor product names with internal naming conventions, facilitating accurate comparisons and seamless integration into subsequent analytical workflows.

Description Generation Bot: The Description Generation Bot was developed to create detailed, engaging, and accurate descriptions for competitor products. These descriptions were generated using the product name and additional attributes such as category, brand. The enriched dataset served as input for embedding processes, enhancing the semantic understanding of the product data.

For the script (`DescriptionGenerationBot.py`) refer to Appendix A.

Prompt Design: The bot was guided by the following **few-shot learning prompt**. For the complete template, see the prompt template for the Description Generation Bot.

Task and Output Format: The bot was tasked with generating a concise and engaging product description based on the product name and additional attributes. The output format followed a clear structure, as illustrated in the examples above:

- **Input:** Name: Red Bull Energy Drink (EINWEG) (0.25 l)
- **Output:** Red Bull Energy Drink, a popular energy beverage, available in a 0.25L can. Known for its caffeine content and refreshing taste, it is ideal for boosting energy and focus.

The descriptions generated by the bot adhered to the following principles:

- Highlighted the unique features and benefits of the product.
- Used a consistent and engaging tone to make the descriptions more user-friendly.
- Avoided redundancy by not repeating information already conveyed in the product name.

Impact on Data Enrichment: These preprocessing steps transformed the raw data into a standardized and enriched dataset, significantly enhancing its suitability for embedding processes. By incorporating semantic richness into the dataset, the bot facilitated more accurate vectorization and downstream analysis.

3.4 Embedding Models and Semantic Search Implementation

3.4.1 Embedding Models Selection

In this thesis, we evaluated two multilingual embedding models: **multilingual-e5-large** and **jina-embeddings-v3**. Both models are designed for multilingual text embedding and are capable of handling multiple languages efficiently. Each model has approximately **530 million parameters**, making them lightweight yet powerful options for embedding-based retrieval tasks.

The *Massive Text Embedding Benchmark (MTEB)* (Muennighoff et al., 2022) leaderboard on Hugging Face was used as the primary reference for selecting the models. The **datasets used in this thesis** (*Competition_data.xlsx* and *Internal_products.csv*) are predominantly in **German**, which makes retrieval performance in German a crucial evaluation metric. For similarity-based retrieval systems, selecting an embedding model with high retrieval accuracy is essential (Muennighoff et al., 2022).

The table below summarizes the key characteristics of the top-performing models for retrieval tasks in German, specifically focusing on the **GermanQuAD-Retrieval** benchmark.

Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	GermanQuAD-Retrieval
`jina-embeddings-v3`	572	**2.13**	1024	**95.46**
`e5-mistral-7b-instruct`	7111	26.49	4096	95.21
`GritLM-7B`	7240	26.97	4096	95.32
`multilingual-e5-large`	560	**2.09**	1024	**94.66**
`multilingual-e5-base`	278	1.04	768	93.93

Table 3.4: Comparison of embedding models based on retrieval performance in German. Data sourced from MTEB (Muennighoff et al., 2022). Highlighted values indicate criteria used for model selection. *Source: Author.*

From Table 3.4, it is evident that **jina-embeddings-v3** and **multilingual-e5-large** exhibit significantly lower memory usage (2.13 GB for `jina-embeddings-v3` and 2.09 GB for `multilingual-e5-large`) compared to larger models such as `e5-mistral-7b-instruct` (26.49 GB) and `GritLM-7B` (26.97 GB). This makes them computationally efficient while maintaining high retrieval performance in **GermanQuAD-Retrieval** (Muennighoff et al., 2022).

Although `multilingual-e5-base` has the lowest memory usage (1.04 GB), its retrieval performance (93.93) is notably lower than that of `multilingual-e5-large` (94.66). The trade-off between computational efficiency and retrieval accuracy was a key consideration in selecting the optimal models for this bachelor thesis.

By leveraging models with **lower computational costs and strong retrieval capabilities**, we ensured that embeddings could be generated **efficiently** without compromising retrieval quality.

3.4.2 Embedding Models Usage

To evaluate the effectiveness of different multilingual embedding models for product representation, we conducted experiments using `jina-embeddings-v3` and `multilingual-e5-large`. The primary objective was to assess how different levels of product information contribute to the quality of embeddings and their usability in a vector database (`PostgreSQL`).

Embedding Generation Process

The embedding generation process consists of three main steps:

1. **Preprocessing product information**: Preparing product data by handling missing values, removing unnecessary characters, and structuring text appropriately.
2. **Generating embeddings**: Encoding product text into dense numerical vectors using the selected embedding models.
3. **Storing embeddings in a vector database**: Organizing the embeddings in -PostgreSQL with the pgvector extension for efficient retrieval.

Step 1: Preprocessing Product Information Before generating embeddings, the product data undergoes a series of preprocessing steps to ensure consistency and relevance. The function `preprocess_for_embedding` is responsible for structuring the input text by handling missing values and ensuring that redundant information is removed. The code snippet in Listing 3.1 demonstrates these preprocessing steps.

```
1  def preprocess_for_embedding(prod_data, ex_keys):
2      """Prepares text for embedding by handling missing values and \
           structuring input."""
3
4      # Replace NaN values with empty strings
5      for k, v in prod_data.items():
6          if pd.isna(v):
7              prod_data[k] = ""
8
9      prod_data["brand_name"] = prod_data["name"]
10
11     # Remove brand name from product name to avoid redundancy
12     prod_data["name"] = re.sub(
13         re.escape(prod_data["brand"]), "", prod_data["name"], \
               flags=re.IGNORECASE
14     )
```

Listing 3.1: Preprocessing product data before embedding generation *Source: Author.*

The `preprocess_for_embedding` function performs the following key operations:

- **Handling Missing Values:** The function iterates through all product attributes and replaces any missing (`NaN`) values with an empty string to ensure that the embedding model does not encounter null inputs.
- **Defining Key Text Fields:** The primary focus is on two columns:
  - `brand_name`: field is created by concatenating the product's brand and its name. This ensures that embeddings capture both the brand identity and product's name.
  - `description`: This field contains the product description.

These preprocessing steps ensure that the text input used for embeddings is structured efficiently, reducing noise and improving the accuracy of similarity comparisons in the vector database.

Step 2: Generating Embeddings The code snippet in Listing 3.2 shows the embedding generation process, where product descriptions and brand names are combined and encoded into dense vector representations.

```
1
2 model = SentenceTransformer("intfloat/multilingual-e5-large")
3
4 def generate_embedding(text):
5     """Generates an embedding vector from text."""
6     return [float(i) for i in model.encode(text)]
```

Listing 3.2: Generating embeddings from product descriptions *Source: Author.*

Step 3: Storing Embeddings in a Vector Database Once the embeddings are generated, they are stored in a `PostgreSQL` database using the `pgvector` extension, which allows for efficient similarity searches. Listing 3.3 illustrates how embeddings are inserted into the database.

```
1
2 emb_cols = ["brand_name", "description"]
3
4 def create_insert_embddings(cur, table_name, product_texts, model):
5     # Get existing product IDs
6     existing_ids = get_existing_product_ids(cur, table_name)
7
8     new_products = {k: v for k, v in product_texts.items() if k not in \
          existing_ids}
9
10    if not new_products:
11        print("No new products to process")
12        return
13
14    print(f"Processing {len(new_products)} new products")
15
16    for k, v in tqdm(new_products.items(), file=sys.stdout):
17        embedding = [float(i) for i in model.encode(v)]
```

```
18          value = cur.mogrify("(%s,␣%s,␣%s)", (k, v, \
                embedding)).decode("utf-8")
19
20          cur.execute(
21              f"INSERT␣INTO␣{table_name}␣(unique_id,␣text,␣embedding)␣VALUES␣\
                    {value}"
22          )
```

Listing 3.3: Storing embeddings in PostgreSQL with pgvector *Source: Author.*

Complete Code Reference For a full implementation of the embedding generation and storage process, refer to the complete script provided in **Appendix A**.

Table 3.5 illustrates an example of how the embeddings are structured in the vector database. The `id` column represents the primary key, `unique_id` is a unique product identifier, and `text` contains the processed product information used for embedding generation. This text typically includes the brand name and product description, ensuring that the embeddings capture meaningful semantic relationships. Finally, the `embedding` column contains the high-dimensional vector representation of the processed product information.

id (PK)	unique_id	text	embedding (vector)
33879	25024_de	brand_name: Dr Bronners 18in1 Naturseife...	[0.0386, -0.0015, -0.0189, ..., 0.0432]
33880	72812_de	brand_name: formoline eiweißdiät Pulver 480g...	[0.0456, 0.0046, -0.0191, ..., 0.0444]
33881	79614_de	brand_name: Ludwigsluster Schweineminutensteaks...	[0.0424, 0.0048, -0.0378, ..., 0.0601]
33882	64936_de	brand_name: Bambino Mio AntiSauerei Windelvlies...	[0.0230, -0.0066, -0.0054, ..., 0.0529]
33883	4969_de	brand_name: Captain Morgan Original Spiced Gold...	[0.0318, 0.0095, -0.0520, ..., 0.0758]

Table 3.5: Example of the final embedding table in the vector database. *Source: Author.*

3.5 Design and Implementation of Semantic Search

3.5.1 Overview of the Semantic Search System

In modern product discovery, keyword-based search techniques often fail to capture the nuances of user intent, leading to suboptimal search results. To address this limitation, a **semantic search system** is designed to retrieve products based on contextual meaning rather than exact keyword matches. The system leverages **high-dimensional embeddings** and **vector-based similarity search** to enhance retrieval performance.

Figure 3.1 illustrates the **end-to-end workflow** of the semantic search system, including data preprocessing, embedding generation, query processing, and retrieval.

The system consists of the following key components:

- **Input Dataset and Preprocessing**
- **Embedding Model and Vector Storage**

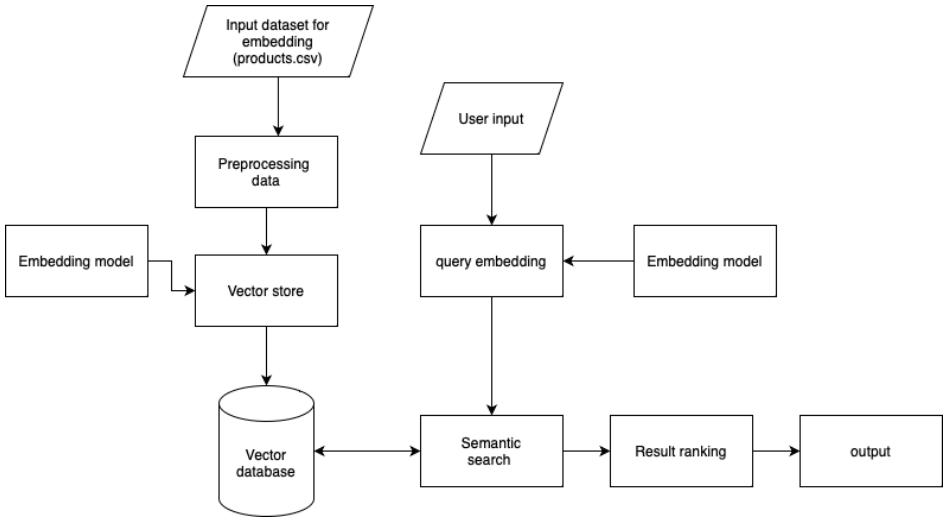

Figure 3.1: Overview of the Semantic Search System. *Source: Author.*

- **Query Embedding**
- **Semantic Search**
- **Result Ranking**
- **Output**

Each of these components plays a crucial role in ensuring **accurate, efficient, and scalable** retrieval of semantically relevant products.

3.5.2 System Components and Workflow

Input Dataset

The foundation of the system is the **product dataset** (`products.csv`), which contains structured product information, including:

- **Product Name** – The official name of the product.
- **Brand** – The manufacturer or supplier associated with the product.
- **Description** – A textual representation of the product's features and attributes.

Since raw product data is often **incomplete** or **inconsistent**, a preprocessing stage is required to standardize the input before generating embeddings.

Preprocessing Data

To improve retrieval accuracy, the raw text undergoes **several preprocessing transformations** before being embedded into vector representations. The key steps include:

1. **Text Normalization**

- Converting text to lowercase.
- Removing special characters and unnecessary symbols.
- Standardizing whitespace and punctuation.

2. **Concatenation of Key Attributes**
   - Combining **product name, brand, and description** into a structured format to ensure rich contextual embeddings.
   - The structured format follows:

     ```
     brand_name: [value]; description: [value]
     ```

3. **Handling Missing Descriptions**
   - For products lacking descriptions, the **Description Generation Bot** (see in Chapter 3) is used to generate meaningful summaries based on available attributes.

By standardizing the text input, we ensure that the **embedding model captures consistent semantic representations** of the products.

Embedding Model The preprocessed product descriptions are transformed into high-dimensional vector representations using a **state-of-the-art embedding model**. In this implementation, we utilize the **Multilingual-e5 Large** embedding model, which is optimized for semantic similarity tasks.

This model generates **dense vector embeddings** where semantically similar products have closely aligned vectors in a multi-dimensional space. The dimensionality is carefully chosen to **balance computational efficiency** and **semantic expressiveness**.

Vector Store The **vector store** acts as a staging area before persisting embeddings in a specialized database. This intermediate step enables:

- Efficient indexing of vectors for quick retrieval.
- Batch updates and real-time storage optimizations.

Vector Database A **vector database** is used to store the embeddings and support high-performance nearest-neighbor searches. Key functionalities include:

- **Efficient similarity search algorithms** such as **Hierarchical Navigable Small World (HNSW)** or **Inverted File Index (IVF)**.
- **Scalable storage** capable of handling millions of vector embeddings.
- **Fast retrieval mechanisms** utilizing optimized index structures.

This database serves as the core storage mechanism that enables rapid semantic matching between queries and stored product data.

Query Processing

User Input The system accepts **user queries** in natural language format, which can range from simple keywords to detailed descriptions. The flexibility of input allows for more intuitive product searches.

Query Embedding The **query embedding** component transforms user input into vector space using the same **embedding model** that was used for the product database. This ensures that both queries and product data exist in the same high-dimensional space, making **semantic comparisons** possible.

Result Ranking and Output Presentation

Result Ranking The retrieved products are **ranked** based on their similarity scores. Higher similarity values indicate a closer semantic match to the query.

Output The output consists of:

- **Product details** (name, brand, description).
- **Similarity scores** to indicate relevance.
- A ranked list of suggested products.

This ensures that users receive the most relevant search results in an intuitive and interpretable format.

3.6 Classifier leveraging an LLM model

3.6.1 Classification Process

After retrieving the most similar products through the semantic search system, it is crucial to determine whether these products are *identical*, *similar*, or *different* from the competitor's product. To achieve this, we implement a classification system using a **Large Language Model (LLM)**, specifically the `gpt-4o` model (OpenAI, 2024).

The classification system is built as an **LLM-powered bot**, referred to as the *Classification Bot*, which is responsible for evaluating and labeling retrieved products based on predefined criteria. These classification criteria are structured within a prompt template (further explained in Prompt Design and Instructions), provides explicit definitions and step-by-step guidelines to help the bot distinguish between identical, similar, or different products.

Given a competitor's product, the semantic search system (introduced in Section 3.5) retrieves a list of potentially matching products from the vector database. However, these retrieved products may not always be exact matches. To address this issue, an LLM-based classification bot is employed to analyze and categorize each retrieved product.

The classification process follows these steps:

1. **Competitor Product Input:** The competitor's product name and attributes are provided as input to the system.
2. **Semantic Search Retrieval:** The system retrieves the most similar products based on their vector embeddings.
3. **LLM-Based Classification:** Each retrieved product is compared against the competitor's product using GPT-4o, guided by a structured classification prompt.
4. **Final Labeling:** The model assigns one of the following categories to each retrieved product:
   - **Identical:** The product is an exact match based on branding, attributes, and packaging.
   - **Similar:** The product has minor differences but serves the same purpose.
   - **Different:** The product is significantly different from the competitor's product.

3.6.2 Prompt Design and Instructions

To ensure accurate classification, the LLM is prompted using a detailed instruction set. The classification logic follows the guidelines outlined in the document *Instructions for Product Analysis and Classification* (Appendix A). These guidelines define the criteria for identifying identical, similar, or different products.

An example prompt template is as follows:

```
prompt_template = {
    "role": "system",
    "content": (
        "### Product Classification System ###\n\n"
        "You are a product analysis expert. Your task is to \
            compare a competitor's product "
        "against a list of retrieved products and classify each \
            as 'Identical', 'Similar', "
        "or 'Different'.\n\n"

        "### Classification Criteria ###\n"
        "- **Identical**: Exact match in brand, manufacturer, \
            attributes, and packaging.\n"
        "- **Similar**: Minor differences but fulfills the same \
            customer need.\n"
        "- **Different**: Substantially different product.\n\n"

```

```
14          "### Instructions ###\n"
15          "Analyze each retrieved product and assign the \
                appropriate classification label."
16      )
17  }
```

Listing 3.4: **Example LLM Prompt for Classification,** ***Source: Author.***

To illustrate the classification process, consider the following example:

- **Competitor Product:** "Dr. Oetker Vanillin Zucker 10 Stück"
- **Semantic Search Results:**
  - "Dr. Oetker Vanillin Zucker 10 Stück"
  - "Dr. Oetker Backin Backpulver 10x16g 0.16 kg"
  - "Dr. Oetker Original Backin 3 Beutel 0.048 kg"

Applying the Classification Bot, the results are:

- "Dr. Oetker Vanillin Zucker 10 Stück" → **Identical** (exact match in brand, attributes, and packaging)
- "Dr. Oetker Backin Backpulver 10x16g 0.16 kg" → **Similar** (same brand and category but different product type)
- "Dr. Oetker Original Backin 3 Beutel 0.048 kg" → **Similar** (closely related product but differs in size and purpose)

Products classified as **Identical** are exact matches in brand, attributes, and packaging, ensuring they are the same product. The **Similar** category includes products that share the same brand or function but differ in aspects like size or composition. The **Different** category (not shown in this example) would include products that serve entirely distinct purposes and cannot be used interchangeably.

The use of an LLM-based classification system provides several benefits:

- **Scalability:** Can process large volumes of product comparisons efficiently.
- **Consistency:** Ensures standardized classification across multiple products and categories.
- **Adaptability:** The classification logic can be refined and improved by updating the prompt template.
- **Automation:** Reduces manual effort and enhances productivity in product matching tasks.

By leveraging an advanced LLM, this classification approach improves the accuracy of product matching (see Chapter 4), ultimately enhancing the effectiveness of the semantic search system.

4. Evaluation

4.1 Evaluation of the Semantic Search System

To assess the performance of the semantic search system, we employ **recall and ranking-based metrics**. The primary objective of this evaluation is to determine how effectively the system retrieves the correct internal product when given a competitor's product as input.

In this context, we use the **Top-10 success rate**, which measures how often the correct internal product appears within the top 10 most similar results. This metric is equivalent to **Recall** and serves as a practical indicator of the system's retrieval quality.

4.1.1 Evaluation Methodology

The evaluation process consists of the following steps:

1. **Data Preparation and Evaluation Dataset Construction**
   - First, two datasets were merged:
     - **Internal Product Dataset** (`product_df`) – containing details of products stored in the internal system.
     - **Competitor Product Dataset** – containing records of products offered by competitors.
   - The datasets were joined using the **manufacturer code (EAN_code)**, which serves as a unique identifier for products that are identical across both datasets.
   - This step enabled the construction of the **evaluation dataset**, allowing identification of competitor products with an exact match in the internal database.

   Table 4.1 provides an example of the evaluation dataset after merging, showing the competitor's product details, the corresponding internal product, and the list of similar products retrieved by the semantic search system.

product_name_nielson	EAN_code	internal_product_id	name	similar_product_id
RED BULL NORMAL M CO2 NORMAL 1 0.25 L DS EW	90162565	2528_de	Red Bull Energy Drink (EINWEG) (0.25 l)	[2528_de, 19295_de, 2529_de, 2534_de, 2535_de, ...]
DALLMAYR PRODOMO COFF GEM 500 G PACK	4008167103714	708_de	Dallmayr prodomo gemahlen (0.5 kg)	[708_de, 709_de, 711_de, 707_de, 703_de, 702_de, ...]
MELITTA CAFE AUSLESE COFF GEM 500 G PACK	4002720022261	761_de	Melitta Auslese Filterkaffee klassisch gemahlen	[761_de, 760_de, 92650_de, 762_de, 91349_de, ...]
GORBATSCHOW WODKA 37.5 PRZ 0.7 L	4003310013759	4995_de	Gorbatschow Wodka (0.7 l)	[4995_de, 4994_de, 4993_de, 27097_de, 27096_de, ...]
APEROL ORANGE APERITIF 11 PRZ 0.7 L FL	8002230000319	4954_de	Aperol Aperitivo (0.7 l)	[4954_de, 20621_de, 5248_de, 31825_de, 5008_de, ...]

Table 4.1: Evaluation Table after Merging Internal and Competitor Products, *Source: Author.*

2. **Applying the Semantic Search System**
   - For each competitor product, the semantic search system is queried to retrieve the **top 10 most similar products** from the internal database.
   - The retrieved products are ranked based on their semantic similarity scores.
3. **Measuring Recall**

- If a competitor product has a known exact match(determined via manufacturer code matching), we check whether the **expected internal product ID** appears in the **list of retrieved products**.
- The system is considered successful for that query if the correct product ID is present in the **top 10 returned results**.
- **Recall** is then computed as follows:

$$\text{Recall} = \frac{\text{Number of correctly retrieved product IDs}}{\text{Total number of known identical products}} \tag{4.1}$$

4.1.2 Ranking-Based Evaluation

Beyond Recall, we assess how well the semantic search ranks the correct product within the **top 10 results**. Since we know the exact internal product ID for each competitor product, we determine its position in the retrieved list.

- **Ranking positions:**
  - **Top 1:** The correct product appears in the first position.
  - **Top 5:** The correct product appears within the first five results.
  - **Top 10:** The correct product appears anywhere in the retrieved list.

The percentage of cases where the identical product appears in each ranking category is computed as follows:

$$\text{Top-N Success Rate} = \frac{\text{Number of cases where product appears in Top-N}}{\text{Total number of known identical products}} \tag{4.2}$$

where N represents the threshold (e.g., 1, 5, or 10).

Example Case

To illustrate this evaluation process, consider the following example:

- **Competitor product:** `Red Bull Energy Drink (EINWEG) (0.25 l)`
- **Identified exact match in our system:**
  - After merging the datasets using **manufacturer code**, we confirm that the internal system contains the same product with a known **product ID**.
- **Semantic Search System Output:**
  - The system retrieves the **top 10 most similar products**, returning a ranked list of internal product IDs.
- **Evaluation Results:**
  - If the correct product ID appears in the **top 1**, it is considered a **perfect retrieval**.
  - If it appears within the **top 5**, it is a **highly relevant retrieval**.

- If it is present in the **top 10**, the system has at least **partially succeeded** in retrieving a relevant result.

The combination of Recall and ranking-based metrics provides a comprehensive evaluation of the system's effectiveness in retrieving the most relevant products.

4.2 Evaluation of the Classification Bot

Evaluating the performance of the Classification Bot is crucial for determining how well it automates the task of identifying identical products. For competitor products that provide an **EAN code**, verifying identical matches is straightforward, as we can join datasets and accurately determine whether a corresponding internal product exists. However, for products without an EAN code, identifying identical matches becomes significantly more challenging. A manual review of each product would be costly and time-consuming, making automation necessary.

To address this, we utilize the **Classification Bot** (introduced in Section 3.6) to -autonomously classify retrieved products into `identical`, `similar`, or `different` categories. The evaluation of the Classification Bot is based on **recall** and **precision**.

4.2.1 Evaluation Setup

Using the **evaluation dataset after merging Internal and Competitor Products** (see Section 4.1), we determine the correct internal product for each competitor product. The system retrieves the **top 10 most similar products** using the **Semantic Search System**, ranked from position 1 to 10.

To evaluate the Classification Bot's performance, we compare:

1. The **position of the actual internal product** (our_product_id) in the retrieved `similar_product_id` list;
2. The **position of the 'identical' label** in the `similarity_category` list, which stores the Classification Bot's predictions for each retrieved item.

The `classification_result` column is set to 1 if the `identical` label is assigned to the product at the same index as its actual position in the retrieved list. Otherwise, it is set to 0. For example, in the first row of Table 4.2, the internal product `2528_de` appears in the first position of the retrieved list and is also labeled as `identical` in the first position—thus a correct classification (1). Conversely, the case of `81137_de` results in a mismatch, producing a 0.

The column `identical_count` reflects the number of retrieved products that the Classification Bot labeled as `identical`, enabling later computation of classification precision.

our_product_id	similar_product_id	similarity_category	classification_result	identical_count
2528_de	['2528_de', '19295_de', '2529_de', '2534_de', ...]	[identical]	1	1
708_de	['708_de', '709_de', '711_de', '707_de', '703_de']	[identical]	1	1
761_de	['761_de', '760_de', '92650_de', '762_de', ...]	[identical]	1	1
81137_de	['81137_de', '81139_de', '81138_de', '32619_de', ...]	[similar, similar, different, similar, ...]	0	0
6379_de	['6379_de', '6635_de', '6350_de', '6556_de', ...]	[similar, different, different, different, ...]	0	0

Table 4.2: Sample Output from the Classification Bot Evaluation, *Source: Author.*

If the Classification Bot assigns `identical` to the correct product in the same ranking position as its actual occurrence, then it has successfully classified the product correctly.

4.2.2 Evaluation Metrics

The performance of the Classification Bot is measured using the following two key metrics:

- **Recall**: The proportion of correctly identified identical products. This is computed as:

$$\text{Recall} = \frac{\text{Number of correctly classified identical products (matching order)}}{\text{Total number of products had been classified}} \tag{4.3}$$

- **Precision**: The proportion of products classified as `identical` that were actually correct. This is computed as:

$$\text{Precision} = \frac{\text{Number of correctly classified identical products}}{\text{Total number of products classified as identical by the bot}} \tag{4.4}$$

Recall measures how often the Classification Bot correctly identifies the exact matching product within the list returned by the Semantic Search System. For each competitor product, we determine the correct product ID from the internal database and check its ranking within the top 10 results from the Semantic Search System. The bot's decision is then compared against the actual ground truth (i.e., the correct product ID). If the bot successfully identifies and classifies the correct product as `identical` in the correct position, it is considered a correct prediction.

Precision evaluates the reliability of the bot when it classifies a product as `identical`. It measures how many of the products classified as identical were actually correct. This is crucial because a high precision score indicates that when the bot labels a product as identical, it is highly likely to be correct.

The combination of the **Semantic Search System** and the **Classification Bot** provides an efficient solution for identifying identical products even in cases where EAN codes are unavailable, enhancing the overall reliability of the product matching process.

4.3 Evaluation results and Discussion

4.3.1 Evaluation result of the Semantic Search System

The performance of the semantic search system was evaluated using Recall and ranking metrics. Table ?? presents the key results obtained from the evaluation.

Metric	**Result**
Recall	93.24%
Percentage of times `our_product_id` is present in the top 10`similar_product_id`	93.24%
Percentage of times `our_product_id` is in the top 5 `similar_product_id`	90.20%
Percentage of times `our_product_id` matches the first `similar_product_id`	66.72%

Table 4.3: Evaluation Metrics Results, *Source: Author.*

The evaluation results indicate that the semantic search system performs well in identifying the correct internal product corresponding to a given competitor product. The overall Recall of **93.24%** suggests that, in most cases, the system successfully retrieves a relevant product.

Additionally, the ranking results show that the correct internal product appears in the retrieved list **93.24%** of the time, within the **top 5 results 90.20%** of the time, and in the **first position (ranked as the most similar product) 66.72%** of the time. This means that in **two-thirds of all cases, the system ranks the correct product as the most relevant match**.

These results demonstrate that the system is highly effective in prioritizing and ranking relevant product matches, making it a valuable tool for automated product comparison and catalog integration.

4.3.2 Evaluation Results of the Classification Bot

To evaluate the Classification Bot's effectiveness in identifying identical products from semantic search results, three different large language models (LLMs) were used as the underlying engine: `GPT-4o` (OpenAI, 2024), `Deepseek V3` (DeepSeek AI, 2024), and `Gemini 2.0 Flash` (Google DeepMind, 2024). Each model was tested on the same evaluation dataset consisting of **1,188 product entries**, and their performance was assessed in terms of **recall**, **precision**, **runtime**, and **cost**.

GPT-4o achieved the highest overall performance. It reached a **recall** of **74%**, meaning that in **871 out of 1,184 cases**, it correctly identified and classified the correct product as `identical`. Its **precision** score was **90%**, indicating that when it labeled a product as `identical`, it was correct in **871 out of 969 cases**. The total inference time was approximately **4 hours**, and the processing cost was estimated at **11 USD**.

Deepseek V3 followed closely with a **recall** of **71%**—**842 correct classifications**—and a **precision** of **87%** from **842 out of 968 cases**. Its runtime was slightly longer at **5 hours**, but it was significantly more cost-efficient, incurring a total cost of only **5 USD**.

Gemini 2.0 Flash demonstrated the shortest processing time at just **35 minutes** and the lowest cost of **1 USD**. Despite this efficiency, its **recall** was the lowest at **66%** (with **782 correct classifications**), and it achieved a **precision** of **88%**, being correct in **782 out of 890 cases** where it predicted a product as `identical`.

Table 4.4 provides a summary of the benchmark results for all three models.

Model Name	Recall	Precision	Runtime	Cost (USD)
GPT-4o	0.74	0.90	4h	11
Deepseek V3	0.71	0.87	5h	5
Gemini 2.0 Flash	0.66	0.88	35 min	1

Table 4.4: Benchmark Comparison of LLMs for Product Classification (1,188 Products), *Source: Author.*

These results demonstrate that `GPT-4o` is the most accurate and reliable model for the classification task, particularly in scenarios where **precision and the accurate identification of identical products** are of highest importance. Its superior performance makes it an ideal choice for applications that require high confidence in product pairing. Conversely, for organisations that prioritise **speed and cost-efficiency**, `Gemini 2.0 Flash` emerges as a compelling option, offering rapid inference times at significantly lower operational cost. Meanwhile, `Deepseek V3` presents a balanced compromise between performance and resource usage, making it suitable for general-purpose deployments where moderate trade-offs are acceptable. The ability of the Classification Bot to seamlessly integrate with these diverse LLMs allows for **flexible deployment strategies**, enabling businesses to align model selection with their specific operational priorities and constraints.

Conclusion

The evaluation of the developed two-stage system, comprising a semantic search component followed by an LLM-driven classification bot, demonstrates a robust and adaptable solution for product matching in e-commerce.

The semantic search module exhibited strong performance, achieving a recall of 93.24%. This result confirms the system's ability to consistently retrieve relevant internal products when provided with external product descriptions. Furthermore, the correct product was found among the top five results in over 90% of the cases and ranked first in approximately two-thirds of all queries. These findings underline the semantic search module's strength not only in retrieving relevant results but also in prioritising the most appropriate candidates.

The second stage, powered by a classification bot utilising various large language models (LLMs), provided additional precision. GPT-4o achieved the highest performance, with a recall of 74% and a precision of 90%, making it suitable for accuracy-critical applications. Gemini 2.0 Flash offered exceptional computational efficiency, completing the task in just 35 minutes at minimal cost, albeit with a lower recall. Deepseek V3 offered a balanced trade-off between speed, accuracy, and cost. This comparative analysis illustrates that the optimal LLM depends on specific business priorities, such as whether the focus lies on precision, response time, or computational efficiency. The modularity of the system allows for the dynamic selection of LLMs, making the solution highly adaptable to varying operational requirements.

Limitations and Future Work. While the system shows promising results, several limitations remain. First, the current implementation is tailored to German-language product data and has not yet been extended to support multilingual use cases. Adapting the system for additional languages would increase its generalisability and business applicability. Second, the system currently embeds all products regardless of update frequency, which may become computationally inefficient as product catalogs grow. Future iterations could optimise performance by embedding only newly added products on a scheduled basis (e.g., weekly or monthly), thereby reducing unnecessary computation and cost.

Additionally, although the classification stage achieves acceptable recall (averaging around 70% across all tested LLMs), there remains room for improvement. One promising avenue is to incorporate image-based similarity as a complementary signal. By combining textual matching (based on name and description) with image comparison, the system may better capture semantic equivalence. This could be achieved through the development of a dedicated computer vision model or by leveraging the vision capabilities of modern multimodal LLMs such as GPT-4o, Gemini, or Deepseek. Images could be obtained via web scraping and stored using URLs, allowing a vision-enhanced classification bot to evaluate visual similarity. While this enhancement could improve recall and matching accuracy, it would also increase computational and operational costs, necessitating further efficiency optimisations.

In conclusion, the proposed two-stage architecture offers a scalable and modular solution for semantic product matching. It successfully combines high-recall retrieval with adaptable classification, supporting more accurate product data integration and competitive analysis within modern e-commerce ecosystems. With targeted improvements, the system holds strong potential for even broader deployment and higher performance in real-world applications.

List of references

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent dirichlet allocation [Accessed: 2025-04-23]. http://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf

Bromley, J., Guyon, I., LeCun, Y., Sackinger, E., & Shah, R. (1994). Signature verification using a "siamese" time delay neural network. *Advances in Neural Information Processing Systems (NeurIPS)*, *6*, 737–744.

Chapelle, O., & Chang, Y. (2011). Yahoo! learning to rank challenge overview. *Proceedings of the Learning to Rank Challenge*, 1–24.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using rnn encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*. https://arxiv.org/abs/1406.1078

Covington, P., Adams, J., & Sargin, E. (2016). *Deep neural networks for youtube recommendations*. https://www.researchgate.net/figure/YouTube-recommendation-system-Covington-et-al-2016_fig2_341804453

DataStax. (2023). Real-world applications of cosine similarity [Accessed: 2025-04-06]. https://www.datastax.com/guides/real-world-applications-of-cosine-similarity

DeepSeek AI. (2024). Deepseek-vl: A family of vision-language models [Accessed: February 2, 2025]. https://deepseek.ai

Deng, J. (2022). How customer behavior has transformed as e-commerce has developed in the digital economy. https://www.researchgate.net/publication/366335386_How_customer_behavior_has_transformed_as_e-commerce_has_developed_in_the_digital_economy

Goldberg, Y., & Levy, O. (2014). Word2vec explained: Deriving mikolov et al.'s negative-sampling word-embedding method. *arXiv preprint arXiv:1402.3722*. https://arxiv.org/abs/1402.3722

Google Cloud. (2024). What is natural language processing? [Accessed: 2025-04-19]. https://cloud.google.com/learn/what-is-natural-language-processing

Google DeepMind. (2024). Gemini 2: Next-generation ai models from google [Accessed: February 2, 2025]. https://deepmind.google/technologies/gemini/#gemini-2

Henaff, M., Bruna, J., & LeCun, Y. (2015). Deep convolutional networks on graph-structured data [Accessed: May 6, 2025]. *Proceedings of the International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1506.05163

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*. https://www.bioinf.jku.at/publications/older/2604.pdf

Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing (3rd ed. draft)* [Accessed: May 6, 2025]. https://web.stanford.edu/~jurafsky/slp3/

Kim, Y. (2014). Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*. https://arxiv.org/abs/1408.5882

Koroteev, M. (2021). Bert: A review of applications in natural language processing and understanding. https://arxiv.org/abs/2103.11943

Kuzi, S., Shtok, A., & Kurland, O. (2017). Remedies against the vocabulary gap in information retrieval [Accessed: 2025-04-22]. https://arxiv.org/abs/1711.06004

Li, C., et al. (2020). Scaling e-commerce search: Product retrieval in a million scale catalog [Accessed: 2025-04-22]. https://arxiv.org/abs/2006.11917

Liu, T.-Y. (2009). Learning to rank for information retrieval [Accessed: 2025-04-23]. https://didawiki.cli.di.unipi.it/lib/exe/fetch.php/magistraleinformatica/ir/ir13/1_-_learning_to_rank.pdf

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Mantrala, M. K., Levi, M., Singh, S. S., & Srinivasan, S. (2009). Why is assortment planning so difficult for retailers? a framework and research agenda. *Journal of Retailing*, *85*(1), 71–83. https://doi.org/10.1016/j.jretai.2008.11.006

McKinsey & Company. (2023). What is e-commerce? [Accessed: 2025-04-18]. https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-e-commerce

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. https://arxiv.org/abs/1301.3781

Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2022). Mteb: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*. https://doi.org/10.48550/ARXIV.2210.07316

Nogueira, R., & Cho, K. (2019). Passage re-ranking with bert [arXiv preprint arXiv:1901.04085].

OpenAI. (2024). Gpt-4o: Openai's latest multimodal model [Accessed: February 2, 2025]. https://openai.com/research/gpt-4o

Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543. https://aclanthology.org/D14-1162/

Price2Spy. (2023a). Competitor product assortment analysis: Definition, strategy & benefits [Accessed: 2025-04-06]. https://www.price2spy.com/blog/competitor-product-assortment-analysis/

Price2Spy. (2023b). How semantic search enhances e-commerce conversion rates [Accessed: 2025-04-22]. https://www.price2spy.com/blog/how-semantic-search-boosts-ecommerce-sales/

Prisync. (2023). What is an ean code and why it matters in e-commerce [Accessed: April 18, 2025]. https://prisync.com/blog/ean-code/

Raiaan, M. A. K., Mukta, M. S. H., Fatema, K., Fahad, N. M., Sakib, S., Mim, M. M. J., Ahmad, J., Ali, M. E., & Azam, S. (2024). A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access*, *12*, 26839–26874. https://doi.org/10.1109/ACCESS.2024.3365742

Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4), 333–389.

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). Facenet: A unified embedding for face recognition and clustering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 815–823. https://doi.org/10.1109/CVPR.2015.7298682

ShipBob. (2023). Product assortment: What it is & how to master it [Accessed: 2025-04-06]. https://www.shipbob.com/blog/product-assortment/

Tóth, S., Wilson, S., Tsoukara, A., Moreu, E., Masalovich, A., & Roemheld, L. (2022). End-to-end multi-modal product matching in fashion e-commerce [Accessed: May 6, 2025]. *Proceedings of the ACM Conference on Recommender Systems (RecSys)*. https://arxiv.org/abs/2209.14567

Turing.com. (2023). A comprehensive guide on word embeddings in nlp [Accessed: 2025-04-19]. https://www.turing.com/kb/guide-on-word-embeddings-in-nlp

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*.

Wang, Y., Sun, Y., Fang, Q., & Guo, L. (2018). Feature contribution to semantic item similarity for personalized recommendation. *Computer Science Publications*. https://scholar.uwindsor.ca/cgi/viewcontent.cgi?article=1068&context=computersciencepub

Wiemer-Hastings, P. (2004). Latent semantic analysis [Accessed: 2025-04-23]. https://reed.cs.depaul.edu/peterh/papers/Wiemer-HastingsEnc-LSA.pdf

Zhang, S., et al. (2020). Towards personalized and semantic retrieval: An end-to-end solution for e-commerce search via embedding learning [Accessed: 2025-04-22]. https://arxiv.org/abs/2006.02282

Appendices

A. Appendix: Electronic Attachments

All supplementary files, such as code scripts and the complete datasets, are provided in the electronic attachments submitted in InSIS. They are listed below:

- `NameStandardizationBot.py`
  Python script implementing the Name Standardization Bot, responsible for harmonising inconsistent product naming conventions.
- `create_embeddings.py`
  Python script used to generate vector embeddings from product information using a multilingual transformer model.
- `DescriptionGenerationBot.py`
  Source code for the bot designed to automatically generate missing product descriptions based on available metadata.
- `similarity_categorization.ipynb`
  The file use LLM's response to extract the classification label (Identical, Similar, or Different)
- `Semantic_search.ipynb`
  Jupyter Notebook for executing the semantic search pipeline. It retrieves the top similar internal products for each competitor product and generates a result table.
- `classification_input_data.csv`
  A pre-processed dataset derived from semantic search results. It serves as the input for the Classification Bot.
- `few_shots.xlsx`
  Excel file containing few-shot learning examples used to enhance the performance of the Name Standardization Bot.
- `PromptTemplateNameStandardization.doc`
  Complete prompt template provided to the Name Standardization Bot for guiding the renaming process.
- `PromptTemplateDescriptionGeneration.doc`
  Complete prompt template used by the Description Generation Bot to synthesise missing product descriptions.
- `Competition_data.xlsx`
  Excel file containing raw product data collected from competitor platforms.
- `Internal_products.csv`
  CSV file containing a subset of internal product data used for training and evaluation purposes.
- `Instructions_for_Product_Analysis_and_Classification.md`
  Markdown document detailing the full instruction set and classification criteria used by the Classification Bot.

B. Appendix: Guidelines for Executing Code Scripts

- `read_me.md`
  This file provides detailed instructions for executing the scripts included with this thesis. It outlines the required environment setup, dependencies, and step-by-step usage procedures to reproduce the results presented in the main text.